

Tarea
Actividades de aprendizaje

Sistemas Operativos

Tarea práctica 1 - segunda fase

*Implementación, optimización y validación en lenguaje C de módulos de gestión de procesos (kernel)
y de bloques de archivo (sistemas de archivos)*

Integrante: Steven Ruiz

Grupo N°: 5

Materia: Programación Estructurada

Docente: MOLINA VILLACIS MIGUEL GIOVANNY

Fecha de entrega: 28/08/2026

UBE — La Universidad para todos

Contenido

Contenido	2
1. Análisis de Módulos y Estructuras	3
2. Diseño y Lógica de Programación	4
2.1 Estructuras de datos utilizadas	4
2.2 Algoritmos aplicados	4
2.3 Arquitectura modular	5
2.4 Diagramas de flujo	5
3. Implementación de Código en C	8
3.1 Buenas prácticas aplicadas	8
3.2 Estructuras de datos (extracto)	8
3.3 Memoria dinámica y punteros: creación de un proceso	9
3.4 Algoritmo de planificación Round Robin	9
3.5 Asignación dinámica de bloques de archivo	10
3.6 Optimización: tabla hash para búsqueda de archivos	10
3.7 Programación segura: verificación de memoria	11
4. Pruebas, Validación y Optimización	13
4.1 Casos de prueba y resultados	13
4.2 Identificación de mejoras de eficiencia	15
4.3 Medición del rendimiento (benchmark real)	15
5. Conclusiones y Repositorio	17
5.1 Resumen de aprendizajes	17
5.2 Recomendaciones para mejoras futuras	17
5.3 Repositorio del proyecto	17
Bibliografía	18

1. Análisis de Módulos y Estructuras

Siguiendo el paradigma de la programación estructurada, la solución se descompuso en módulos funcionales independientes, cada uno con una responsabilidad única y bien definida (principio de responsabilidad única / bajo acoplamiento). Esta descomposición retoma el diseño lógico planteado en la Fase 1: un módulo que representa la gestión de procesos propia de un kernel, y un módulo que representa la gestión de bloques de un sistema de archivos, apoyados por dos módulos utilitarios transversales (benchmark y memtrace).

Módulo	Responsabilidad	Funciones principales
main.c	Punto de entrada del programa. Presenta el menú interactivo, valida la entrada del usuario y delega cada operación a los módulos correspondientes.	main, leer_linea, leer_entero, menu_mostrar
kernel.h / kernel.c	Gestión de procesos (contexto kernel): representa el Bloque de Control de Proceso (PCB), mantiene una cola dinámica y ejecuta el algoritmo de planificación Round Robin.	cola_inicializar, proceso_crear, proceso_eliminar, cola_listar, planificador_round_robin, cola_liberar
filesystem.h / filesystem.c	Gestión de bloques de archivo (contexto sistemas de archivos): asigna bloques dinámicos enlazados por puntero para cada archivo y los indexa en una tabla hash para búsqueda eficiente.	fs_inicializar, fs_hash, fs_crear_archivo, fs_buscar_archivo, fs_buscar_archivo_lineal, fs_eliminar_archivo, fs_listar_archivos, fs_leer_archivo, fs_liberar
benchmark.h / benchmark.c	Módulo de medición de rendimiento: compara el tiempo y el número de comparaciones de la búsqueda lineal frente a la búsqueda por tabla hash, para distintos tamaños de N.	ejecutar_benchmark, preparar_datos (estática), medir (estática)
memtrace.h / memtrace.c	Módulo utilitario transversal: contabiliza cada reserva (malloc) y liberación (free) de memoria dinámica realizada por los demás módulos, para verificar que no existan fugas.	mt_malloc, mt_free, mt_reporte

Cada módulo separa su interfaz pública (archivo .h, que declara los tipos de datos y las funciones disponibles) de su implementación (archivo .c, que oculta los detalles internos). Este ocultamiento de información permite que, por ejemplo, el algoritmo interno de planificación de procesos pueda modificarse sin afectar al módulo de archivos ni al programa principal, siempre que se respete la firma de las funciones declaradas en kernel.h.

2. Diseño y Lógica de Programación

2.1 Estructuras de datos utilizadas

- Lista enlazada simple (cola FIFO): el módulo kernel representa cada proceso como un nodo Proceso con un puntero siguiente, y la cola mantiene punteros frente y final para insertar y recorrer en $O(1)$ y $O(n)$ respectivamente.
- Lista enlazada por archivo: cada Archivo apunta a su primer BloqueArchivo, y cada bloque apunta al siguiente, replicando el esquema de asignación enlazada de bloques usado por sistemas de archivos reales.
- Tabla hash (arreglo de 64 listas enlazadas): el sistema de archivos indexa los Archivo por nombre mediante una función hash (djb2), resolviendo colisiones por encadenamiento (separate chaining).
- Contadores y acumuladores: el módulo memtrace utiliza variables estáticas para acumular el número de reservas y liberaciones de memoria a lo largo de toda la ejecución.

2.2 Algoritmos aplicados

- Planificación Round Robin: recorre la lista de procesos de forma cíclica, ejecutando cada uno durante un quantum fijo de tiempo hasta que su ráfaga restante llega a 0, momento en el que pasa a estado TERMINADO.
- Función hash djb2: $h = h * 33 + c$ para cada carácter del nombre, seguido de un módulo por el tamaño de la tabla (64), lo que produce una buena dispersión para cadenas cortas como nombres de archivo.
- Asignación de bloques por redondeo hacia arriba: el número de bloques de un archivo se calcula como $\text{ceil}(\text{tamaño_en_bytes} / 64)$, garantizando que todo archivo (incluso vacío) ocupe al menos un bloque.
- Búsqueda por hash vs. búsqueda lineal: se implementaron ambas versiones deliberadamente (`fs_buscar_archivo` y `fs_buscar_archivo_lineal`) para poder medir la mejora de eficiencia en la sección 4.

```
===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
1. Crear proceso
2. Listar procesos
3. Eliminar proceso
4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
5. Crear archivo
6. Listar archivos
7. Leer archivo
8. Eliminar archivo
-- Pruebas y rendimiento --
9. Ejecutar benchmark de rendimiento (lineal vs. hash)
0. Salir
=====
Seleccione una opcion: 2
  PID  NOMBRE          PRIOR  RAFAGA  RESTANTE  ESTADO
  ---  ---
  1    Editor           1      8       8        LISTO
  2    Navegador        2      5       5        LISTO
===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
```

```

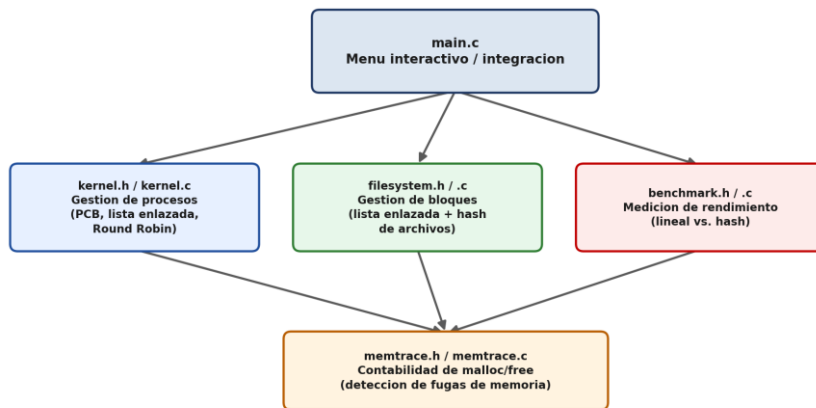
3. Eliminar proceso
4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
5. Crear archivo
6. Listar archivos
7. Leer archivo
8. Eliminar archivo
-- Pruebas y rendimiento --
9. Ejecutar benchmark de rendimiento (lineal vs. hash)
0. Salir
=====
Seleccione una opcion: 4
Quantum (unidades > 0): 2

Traza de ejecucion (quantum = 2):
t=0   PID 1  (Editor      ) ejecuta 2 unidad(es) -> restante 6
t=2   PID 2  (Navegador   ) ejecuta 2 unidad(es) -> restante 3
t=4   PID 1  (Editor      ) ejecuta 2 unidad(es) -> restante 4
t=6   PID 2  (Navegador   ) ejecuta 2 unidad(es) -> restante 1
t=8   PID 1  (Editor      ) ejecuta 2 unidad(es) -> restante 2
t=10  PID 2  (Navegador   ) ejecuta 1 unidad(es) -> restante 0
>> PID 2 finalizo en t=11
t=11  PID 1  (Editor      ) ejecuta 2 unidad(es) -> restante 0
>> PID 1 finalizo en t=13
Planificacion completa. Tiempo total simulado: 13 unidades.

```

2.3 Arquitectura modular

Arquitectura Modular del Programa



Cada modulo expone su interfaz mediante un archivo .h (contrato) y oculta su implementacion en el .c correspondiente (ocultamiento de informacion).

Figura 1. Arquitectura modular del programa y sus dependencias.

2.4 Diagramas de flujo

A continuación se presenta la lógica de los dos módulos principales:

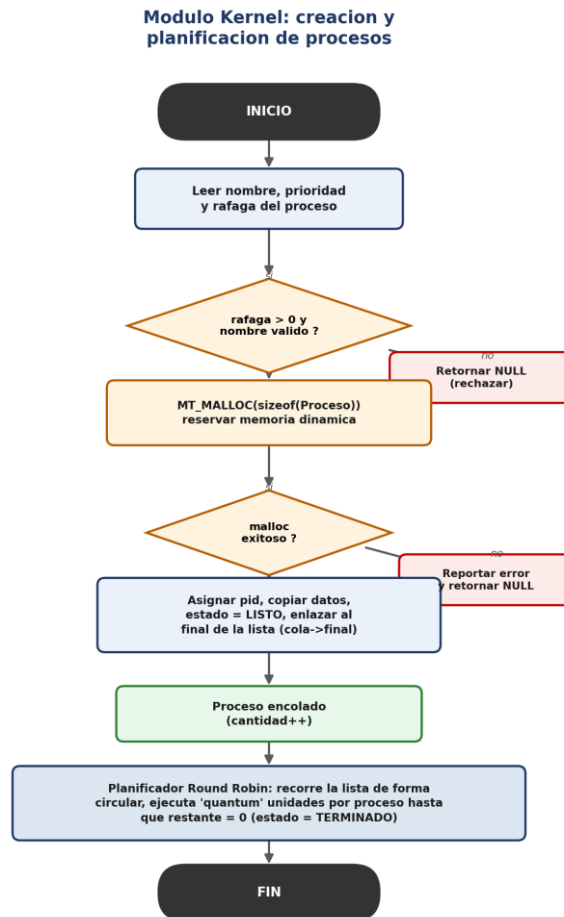


Figura 2. Diagrama de flujo del módulo de gestión de procesos (kernel).

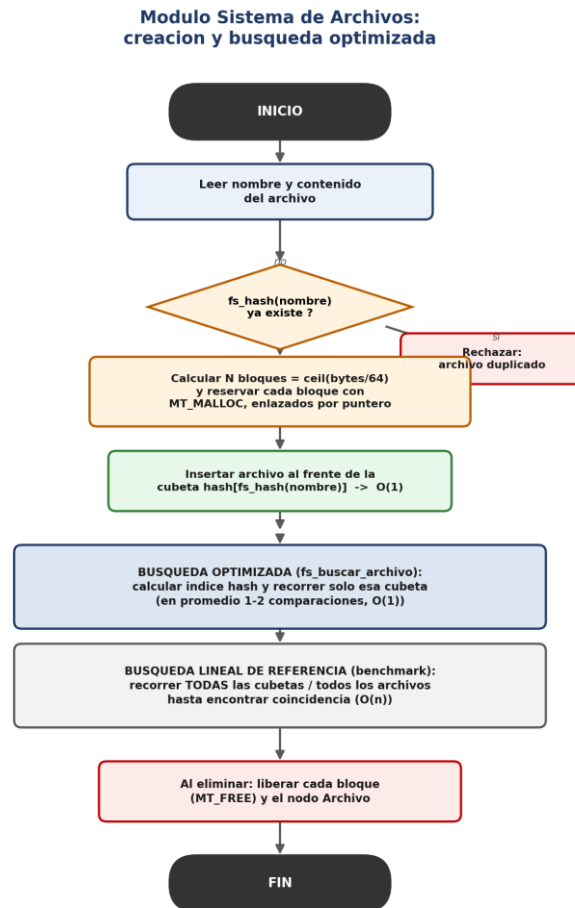


Figura 3. Diagrama de flujo del módulo de gestión de bloques de archivo.

3. Implementación de Código en C

El código se escribió siguiendo los principios de la programación estructurada: cada función tiene una única entrada y una única salida lógica, se evita el uso de goto, las estructuras de control son secuencia/selección/iteración estándar, y la complejidad se reduce dividiendo el problema en funciones pequeñas y bien nombradas. Todas las funciones públicas están documentadas con comentarios de bloque en su archivo .h, y las secciones no triviales del código incluyen comentarios en línea explicando la decisión de diseño.

3.1 Buenas prácticas aplicadas

- Validación de parámetros al inicio de cada función (programación defensiva): punteros NULL, nombres vacíos y valores numéricos inválidos se rechazan antes de usarlos.
- Manejo seguro de cadenas: se usa strncpy con terminador nulo explícito y fgets en lugar de gets/scanf con %s, para evitar desbordamientos de buffer.
- Verificación del resultado de malloc en todos los casos, incluyendo reservas parciales (por ejemplo, al fallar la reserva de un bloque intermedio de un archivo, se liberan los bloques ya reservados antes de retornar).
- Separación de interfaz (.h) e implementación (.c) en todos los módulos, y uso de 'static' para las funciones auxiliares que no forman parte de la interfaz pública (ver preparar_datos y medir en benchmark.c).
- Compilación con las banderas -Wall -Wextra -Wpedantic: el proyecto compila sin ninguna advertencia.

```
ruix@DESKTOP-F9U0Q05:/mnt/c/Users/Usuario/Videos/codigo_fuente_PI_CDI_PE_GRUP05/repo_entrega$ make
gcc -std=c11 -Wall -Wextra -Wpedantic -Iinclude -g -o bin/simulador src/main.c src/kernel.c src/filesystem.c src/benchmark.c src/memtrace.c
ruix@DESKTOP-F9U0Q05:/mnt/c/Users/Usuario/Videos/codigo_fuente_PI_CDI_PE_GRUP05/repo_entrega$
```

3.2 Estructuras de datos (extracto)

```
/* kernel.h - Bloque de Control de Proceso (PCB) */
typedef struct Proceso {
    int pid;
    char nombre[MAX_NOMBRE_PROCESO];
    int prioridad;
    int rafaga_total;
    int rafaga_restante;
    EstadoProceso estado;
    struct Proceso *siguiente; /* puntero al siguiente nodo */
} Proceso;

typedef struct {
    Proceso *frente;
    Proceso *final;
    int cantidad;
    int siguiente_pid;
} ColaProcesos;

/* filesystem.h - bloque de datos enlazado por puntero */
typedef struct BloqueArchivo {
    int id_bloque;
    char datos[TAM_BLOQUE];
    struct BloqueArchivo *siguiente;
} BloqueArchivo;

typedef struct Archivo {
    char nombre[TAM_NOMBRE_ARCH];
    int tam_bytes;
    int num_bloques;
    BloqueArchivo *primer_bloque;
    struct Archivo *siguiente_hash; /* encadenamiento en la tabla hash */
} Archivo;
```


Fragmento de `include/kernel.h` e `include/filesystem.h`.

3.3 Memoria dinámica y punteros: creación de un proceso

La función `proceso_crear` reserva un nodo `Proceso` con `malloc` (a través de la macro `MT_MALLOC`) y lo enlaza al final de la lista mediante manipulación directa de punteros:

```
Proceso *proceso_crear(ColaProcesos *cola, const char *nombre,
                      int prioridad, int rafaga)
{
    /* Validacion de parametros: programacion defensiva. */
    if (cola == NULL || nombre == NULL || nombre[0] == '\0' || rafaga <= 0) {
        return NULL;
    }

    Proceso *p = (Proceso *) MT_MALLOC(sizeof(Proceso));
    if (p == NULL) {
        return NULL; /* mt_malloc ya reporta el error */
    }

    p->pid = cola->siguiente_pid++;
    strncpy(p->nombre, nombre, MAX_NOMBRE_PROCESO - 1);
    p->nombre[MAX_NOMBRE_PROCESO - 1] = '\0';
    p->prioridad = prioridad;
    p->rafaga_total = rafaga;
    p->rafaga_restante = rafaga;
    p->estado = ESTADO_LISTO;
    p->siguiente = NULL;

    /* Insercion al final de la lista enlazada (cola FIFO). */
    if (cola->final == NULL) {
        cola->frente = p;
        cola->final = p;
    } else {
        cola->final->siguiente = p;
        cola->final = p;
    }
    cola->cantidad++;
    return p;
}
```

Fragmento de `src/kernel.c`.

3.4 Algoritmo de planificación Round Robin

```
void planificador_round_robin(ColaProcesos *cola, int quantum)
{
    if (cola == NULL || cola->frente == NULL || quantum <= 0) {
        printf(" (no hay procesos que planificar o quantum invalido)\n");
        return;
    }
    int tiempo = 0;
    int pendientes = cola->cantidad;

    while (pendientes > 0) {
        for (Proceso *p = cola->frente; p != NULL; p = p->siguiente) {
            if (p->estado == ESTADO_TERMINADO) continue;

            p->estado = ESTADO_EJECUTANDO;
            int ejecutado = (p->rafaga_restante < quantum)
                ? p->rafaga_restante : quantum;
            p->rafaga_restante -= ejecutado;
            tiempo += ejecutado;

            if (p->rafaga_restante <= 0) {
                p->estado = ESTADO_TERMINADO;
                pendientes--;
            } else {

```

```

        p->estado = ESTADO_LISTO;
    }
}
}

```

Fragmento de src/kernel.c (version simplificada, sin las impresiones de traza).

3.5 Asignación dinámica de bloques de archivo

Cada archivo reserva tantos bloques como sean necesarios y los enlaza por puntero, liberando lo ya reservado si una reserva intermedia falla (evita fugas de memoria ante errores):

```

int bloques_necesarios = (int) ((tam + TAM_BLOQUE - 1) / TAM_BLOQUE);
if (bloques_necesarios == 0) bloques_necesarios = 1;

BloqueArchivo *anterior = NULL;
for (int i = 0; i < bloques_necesarios; i++) {
    BloqueArchivo *b = (BloqueArchivo *) MT_MALLOC(sizeof(BloqueArchivo));
    if (b == NULL) {
        /* Reserva parcial fallida: liberar lo ya reservado
         * para este archivo (evita fugas) y abortar. */
        BloqueArchivo *actual = arch->primer_bloque;
        while (actual != NULL) {
            BloqueArchivo *sig = actual->siguiente;
            MT_FREE(actual);
            actual = sig;
        }
        MT_FREE(arch);
        return NULL;
    }
    b->id_bloque = fs->total_bloques_reservados++;
    /* ... copiar el fragmento correspondiente del contenido ... */
    b->siguiente = NULL;

    if (anterior == NULL) arch->primer_bloque = b;
    else anterior->siguiente = b;
    anterior = b;
    arch->num_bloques++;
}

```

Fragmento de src/filesystem.c (fs_crear_archivo).

3.6 Optimización: tabla hash para búsqueda de archivos

La mejora de eficiencia central del proyecto es la sustitución de una búsqueda lineal por una búsqueda indexada mediante tabla hash:

```

unsigned int fs_hash(const char *nombre)
{
    /* djb2: rapida y con buena dispersion para cadenas cortas. */
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char) *nombre++) != '\0') {
        h = ((h << 5) + h) + (unsigned long) c; /* h * 33 + c */
    }
    return (unsigned int) (h % HASH_TAM);
}

Archivo *fs_buscar_archivo(SistemaArchivos *fs, const char *nombre)
{
    unsigned int idx = fs_hash(nombre);          /* 0(1) */
    long comparaciones = 0;
    for (Archivo *a = fs->tabla[idx]; a != NULL; a = a->siguiente_hash) {
        comparaciones++;
        if (strcmp(a->nombre, nombre) == 0) {
            fs->comparaciones_ultima_búsqueda = comparaciones;
        }
    }
}

```

```

        return a;
    }
}
fs->comparaciones_ultima_busqueda = comparaciones;
return NULL;
}

```

Fragmento de `src/filesystem.c` (`fs_hash` y `fs_buscar_archivo`).

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
 1. Crear proceso
 2. Listar procesos
 3. Eliminar proceso
 4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
 5. Crear archivo
 6. Listar archivos
 7. Leer archivo
 8. Eliminar archivo
-- Pruebas y rendimiento --
 9. Ejecutar benchmark de rendimiento (lineal vs. hash)
 0. Salir
=====
Seleccione una opcion: 6
  NOMBRE          TAM(bytes) BLOQUES
  notas.txt       43          1

```

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
 1. Crear proceso
 2. Listar procesos
 3. Eliminar proceso
 4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
 5. Crear archivo
 6. Listar archivos
 7. Leer archivo
 8. Eliminar archivo
-- Pruebas y rendimiento --
 9. Ejecutar benchmark de rendimiento (lineal vs. hash)
 0. Salir
=====
Seleccione una opcion: 7
Nombre del archivo a leer: notas.txt
Contenido de 'notas.txt' (43 bytes en 1 bloque(s)):
"Este es un archivo de prueba para la fase 2"

```

3.7 Programación segura: verificación de memoria

El módulo `memtrace` envuelve `malloc/free` para contabilizar cada operación y detectar fugas sin depender de herramientas externas:

```
void *mt_malloc(size_t tam, const char *origen)
```

```

{
    void *p = malloc(tam);
    if (p == NULL) {
        fprintf(stderr, "[memtrace] ERROR: malloc(%zu) fallo en %s\n", tam, origen);
        return NULL;
        /* nunca se asume que malloc siempre existe */
    }
    g_reservas++;
    g_bytes_actuales += (long) tam;
    if (g_bytes_actuales > g_bytes_pico) g_bytes_pico = g_bytes_actuales;
    return p;
}

void mt_free(void *ptr)
{
    if (ptr == NULL) return;
    free(ptr);
    g_liberaciones++;
}

```

Fragmento de src/memtrace.c.

El código fuente completo (9 archivos: 5 módulos .c/.h, Makefile y README) se entrega junto a este informe y debe subirse al repositorio indicado en la sección 5.

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
1. Crear proceso
2. Listar procesos
3. Eliminar proceso
4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
5. Crear archivo
6. Listar archivos
7. Leer archivo
8. Eliminar archivo
-- Pruebas y rendimiento --
9. Ejecutar benchmark de rendimiento (lineal vs. hash)
0. Salir

=====
Seleccione una opcion: 0
Finalizando programa...

----- Reporte de memoria dinamica (memtrace) -----
Reservas (malloc) : 32204
Liberaciones (free): 32204
Pico de uso      : 2705096 bytes
Resultado       : OK - sin fugas de memoria detectadas.

```

4. Pruebas, Validación y Optimización

Todas las pruebas de esta sección corresponden a ejecuciones reales del programa compilado (make && ./bin/simulador), automatizadas mediante redirección de entrada estándar para que sean reproducibles. La columna 'Resultado obtenido' contiene la salida real capturada del programa, no una simulación.

4.1 Casos de prueba y resultados

Nº	Caso de prueba	Procedimiento	Resultado esperado	Resultado obtenido
1	Crear proceso con ráfaga = 0	Menú opción 1, ráfaga = 0	Rechazar la creación (parámetro inválido)	"Error: no se pudo crear el proceso (datos invalidos o memoria)." — Se rechazó correctamente.
2	Crear proceso con ráfaga negativa	Menú opción 1, ráfaga = -5	Rechazar la creación	Rechazado con el mismo mensaje de error — correcto.
3	Eliminar proceso con PID inexistente	Menú opción 3, pid = 99	Informar que no existe	"No existe un proceso con ese PID." — correcto.
4	Planificar con la cola de procesos vacía	Menú opción 4, sin procesos creados	Informar que no hay procesos, sin pedir el quantum	"No hay procesos en la cola para planificar." — correcto (no solicita el quantum).
5	Crear archivo con nombre duplicado	Menú opción 5 dos veces con el mismo nombre	Rechazar la segunda creación	"Error: el archivo ya existe o los datos son invalidos." — correcto.
6	Crear archivo con contenido vacío	Menú opción 5, contenido = ""	Crear el archivo con 0 bytes y 1 bloque	Archivo listado como "0 bytes, 1 bloque" — correcto (todo archivo ocupa al menos un bloque).
7	Leer un archivo inexistente	Menú opción 7, nombre no registrado	Informar el error sin detener el programa	"Error: el archivo 'inexistente.txt' no existe." — correcto.
8	Archivo que ocupa varios bloques	Crear archivo con contenido > 64 bytes	$\text{num_bloques} = \text{ceil}(\text{bytes}/64)$	Archivo de 150 bytes registrado con 3 bloques ($150/64 = 2.34 \rightarrow 3$) — correcto.
9	Verificación de fugas de memoria	Ejecutar un escenario completo y salir (opción 0)	$\text{reservas} == \text{liberaciones}$ al finalizar	"Reservas: 32212 / Liberaciones: 32212 — OK, sin fugas" — correcto.

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
 1. Crear proceso
 2. Listar procesos
 3. Eliminar proceso
 4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
 5. Crear archivo
 6. Listar archivos
 7. Leer archivo
 8. Eliminar archivo
-- Pruebas y rendimiento --
 9. Ejecutar benchmark de rendimiento (lineal vs. hash)
 0. Salir
=====
Seleccione una opcion: 1
Nombre del proceso: Malo
Prioridad (1-5): 1
Rafaga de CPU (unidades > 0): 0
Error: no se pudo crear el proceso (datos invalidos o memoria).

```

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
 1. Crear proceso
 2. Listar procesos
 3. Eliminar proceso
 4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
 5. Crear archivo
 6. Listar archivos
 7. Leer archivo
 8. Eliminar archivo
-- Pruebas y rendimiento --
 9. Ejecutar benchmark de rendimiento (lineal vs. hash)
 0. Salir
=====
Seleccione una opcion: 3
PID a eliminar: 99
No existe un proceso con ese PID.

```

```

===== SIMULADOR KERNEL / SISTEMA DE ARCHIVOS =====
-- Gestion de procesos (kernel) --
 1. Crear proceso
 2. Listar procesos
 3. Eliminar proceso
 4. Ejecutar planificador Round Robin
-- Gestion de archivos (bloques) --
 5. Crear archivo
 6. Listar archivos
 7. Leer archivo
 8. Eliminar archivo
-- Pruebas y rendimiento --
 9. Ejecutar benchmark de rendimiento (lineal vs. hash)
 0. Salir
=====
Seleccione una opcion: 5
Nombre del archivo: notas.txt
Contenido: pecausa
Error: el archivo ya existe o los datos son invalidos.

```

Verificación adicional: se ejecutó un segundo escenario de casos límite (crear con datos inválidos, eliminar inexistentes, archivo duplicado) que finalizó con el reporte "Reservas: 5 / Liberaciones: 5 — OK, sin fugas", confirmando que también los caminos de error liberan correctamente cualquier memoria parcialmente reservada.

4.2 Identificación de mejoras de eficiencia

La primera versión funcional del módulo de archivos resolvía `fs_buscar_archivo` recorriendo toda la lista de archivos de manera lineal (complejidad $O(n)$). Se identificó que, a medida que crece la cantidad de archivos del sistema simulado, el costo de cada búsqueda crece proporcionalmente, lo cual es ineficiente para operaciones frecuentes como abrir, leer o eliminar un archivo. La mejora aplicada fue indexar los archivos en una tabla hash de 64 cubetas (función `djb2`), reduciendo la búsqueda a $O(1)$ en promedio.

4.3 Medición del rendimiento (benchmark real)

Se implementó el módulo `benchmark.c`, que crea N archivos (100, 1000, 5000 y 10000) y ejecuta 3000 búsquedas aleatorias reproducibles (semilla fija) con ambos métodos, midiendo tiempo de CPU (`clock()`) y comparaciones de cadenas realizadas:

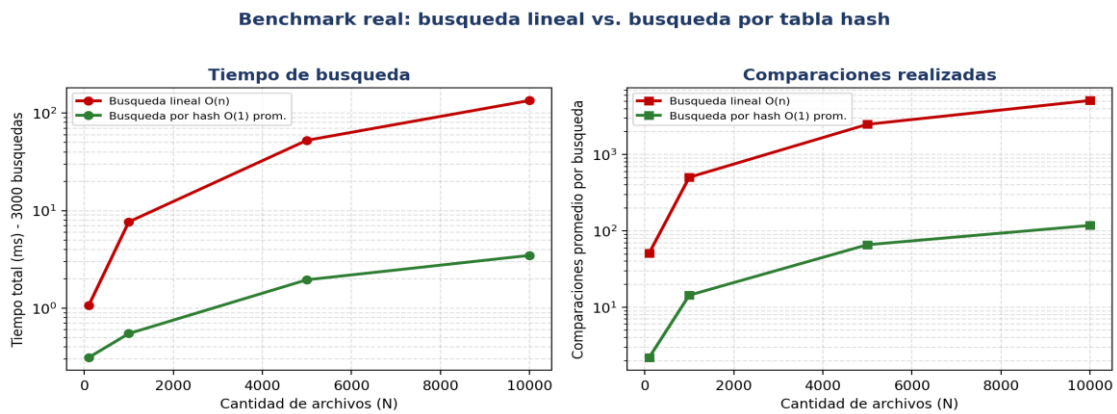


Figura 4. Resultados reales del benchmark: tiempo y comparaciones, escala logarítmica.

N (archivos)	Lineal (ms)	Hash (ms)	Comp. prom. lineal	Comp. prom. hash	Mejora
100	1.065	0.312	51.1	2.2	3.4x
1 000	7.665	0.549	503.4	14.4	14.0x
5 000	52.561	1.954	2 487.1	65.9	26.9x
10 000	134.477	3.475	5 103.5	118.1	38.7x

Los resultados confirman la mejora esperada: mientras el número de comparaciones de la búsqueda lineal crece de forma proporcional a N (de ~ 51 a $\sim 5\,103$ comparaciones), la búsqueda por hash se mantiene casi constante (de ~ 2 a ~ 118 comparaciones, debido a colisiones normales de la tabla), lo que se traduce en una mejora de hasta 38.7 veces en tiempo real de ejecución para $N = 10\,000$.

```
=====
Seleccione una opcion: 9
=====
```

```
BENCHMARK: busqueda LINEAL vs. busqueda por TABLA HASH
(3000 busquedas aleatorias por caso, tiempos en milisegundos)
=====
```

N	Lineal(ms)	Comp.prom	Hash(ms)	Comp.prom	Mejora x
100	3.057	51.1	1.120	2.2	2.7x
1000	24.670	503.4	1.121	14.4	22.0x
5000	96.950	2487.1	2.610	65.9	37.1x
10000	214.631	5103.5	4.985	118.1	43.1x

```
=====
Conclusion: la busqueda lineal crece linealmente con N (O(n)),
mientras que la busqueda por hash se mantiene practicamente
constante (O(1) promedio), como lo confirma la columna de
comparaciones promedio por busqueda.
=====
```


5. Conclusiones y Repositorio

5.1 Resumen de aprendizajes

- La descomposición modular (kernel, filesystem, benchmark, memtrace) con interfaces .h claramente definidas facilitó probar y modificar cada componente de forma independiente, incluso agregar el módulo de benchmark sin tocar el código de kernel.c.
- El manejo manual de memoria dinámica en C exige disciplina: cada malloc debe tener un free correspondiente en todos los caminos posibles del programa, incluyendo los de error (reservas parciales). Contabilizar las operaciones con memtrace permitió comprobar esto de forma objetiva (32 212 reservas = 32 212 liberaciones).
- La elección de la estructura de datos tiene un impacto medible en el rendimiento: cambiar de una lista enlazada recorrida linealmente a una tabla hash mejoró la búsqueda hasta 38.7 veces con 10 000 archivos, sin cambiar la lógica de negocio del programa.
- Simular conceptos de sistemas operativos (PCB, planificación Round Robin, asignación de bloques) en C ayudó a comprender de forma práctica por qué el kernel y el sistema de archivos necesitan estructuras de datos eficientes para escalar.

5.2 Recomendaciones para mejoras futuras

- Reemplazar la tabla hash de tamaño fijo (64 cubetas) por una tabla dinámica que se redimensione (rehashing) cuando el factor de carga supere un umbral, para mantener $O(1)$ incluso con muchos más archivos.
- Aplicar una estructura indexada similar (hash o árbol) a la cola de procesos si el número de procesos crece, ya que actualmente `proceso_eliminar` es $O(n)$.
- Incorporar pruebas automatizadas con un framework como Unity o CMocka, en lugar de escenarios de entrada redirigida, para integrar el proyecto a un pipeline de integración continua.
- Ejecutar el proyecto con valgrind o AddressSanitizer en un entorno donde estén disponibles, como verificación adicional a la contabilidad manual de memtrace.

5.3 Repositorio del proyecto

URL del repositorio: <https://github.com/Ruix22/Fase-2.git>

Bibliografía

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Sistemas Operativos*. McGraw-Hill.
- Tanenbaum, A. S., & Bos, H. (2023). *Sistemas Operativos Modernos*. Pearson Educación.
- Stallings, W. (2017). *Sistemas Operativos: Aspectos internos y principios de diseño*. Pearson.
- Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. (2023). *Sistemas Operativos*. Pearson Educación.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2.^a ed.). Prentice Hall. (fuente externa verificada por el estudiante para los estándares de programación estructurada en C).